Laporan Workshop Desain UI

Disusun untuk Memenuhi Tugas 3 Mata Kuliah

Workshop Desain UI

Di Susun Oleh :

Putri Apriliana 434241022 TI - B4

D4 TEKNIK INFORMATIKA

UNIVERSITAS AIRLANGGA

2025

Modul 3 membuat halaman form menggunakan HTML, CSS, dan JavaScript. Project ini saya membuat 3 file yaitu index.html (struktur), style.css (tampilan), script.js (logika validasi dan interaksi).

1. **Struktur HTML (index.html)**
   a. Deklarasi Dokumen dan penghubung File eksternal

```html
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>Formulir Pendaftaran</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
```

- **<!DOCTYPE html>** — Memberitahu browser bahwa dokumen ini menggunakan standar HTML5. Deklarasi ini wajib ada di baris paling atas agar browser tidak masuk ke mode kompatibilitas lama (quirks mode).
- **<html lang="id">** — Tag pembuka dokumen HTML. Atribut lang="id" mendeklarasikan bahasa halaman sebagai Bahasa Indonesia, penting untuk aksesibilitas dan mesin pencari.
- **<meta charset="UTF-8">** — Menentukan encoding karakter sebagai UTF-8 sehingga karakter khusus seperti huruf berdiakritik dan simbol dapat ditampilkan dengan benar.
- **<meta name="viewport" content="width=device-width, initial-scale=1.0">** — Mengatur viewport agar halaman menyesuaikan lebar layar perangkat dan tidak ter-zoom secara otomatis, krusial untuk tampilan responsif di perangkat mobile.
- **<link rel="stylesheet" href="style.css">** — Menghubungkan file CSS eksternal ke dokumen HTML. Diletakkan di dalam <head> agar style sudah dimuat sebelum browser merender konten halaman sehingga tidak terjadi FOUC (Flash of Unstyled Content).
- **<script src="script.js"></script>** — Menghubungkan file JavaScript eksternal. Diletakkan di akhir <body> (bukan di <head>) agar seluruh elemen HTML sudah selesai diparse dan tersedia di DOM sebelum script dieksekusi. Jika diletakkan di <head>, script akan berjalan sebelum elemen HTML ada sehingga getElementById akan mengembalikan null.

   b. Kontainer card dan header formulir

```html
<div class="card">
    <header class="form-header">
    <h1>Formulir Pendaftaran</h1>
    <p>Silakan isi formulir di bawah ini untuk mendaftar.</p>
    </header>

    <form id="registrationForm" novalidate>
        <div class="form-group">
            <label for="nama">Nama Lengkap</label>
            <input type="text" id="nama" placeholder="Masukkan nama lengkap kamu" />
            <span class="error" id="errNama">⚠ Nama tidak boleh kosong!</span>
        </div>

        <div class="form-group">
            <label for="email">Email</label>
            <input type="email" id="email" placeholder="contoh@email.com" />
            <span class="error" id="errEmail">⚠ Format email tidak valid!</span>
        </div>

        <div class="form-group">
            <label for="umur">Umur</label>
            <input type="number" id="umur" placeholder="Masukkan umur kamu" min="1" max="120" />
            <span class="error" id="errUmur">⚠ Umur harus antara 1 – 120!</span>
        </div>
```

- ** ** — Div pembungkus utama yang bertindak sebagai kartu visual. Class "card" didekorasi di CSS dengan background semi-transparan, border-radius, dan box-shadow untuk efek glassmorphism.
- **<header class="form-header">** — Elemen semantik HTML5 yang menandai bagian kepala dari kartu formulir. Penggunaan <header> lebih tepat secara semantik dibanding biasa karena mendeskripsikan maksud elemen kepada browser dan screen reader.
- **<h1>Formulir Pendaftaran</h1>** — Heading level 1 sebagai judul utama halaman. Setiap halaman sebaiknya memiliki tepat satu <h1> untuk keperluan SEO dan aksesibilitas.
- **<form id="registrationForm" novalidate>** — Elemen form utama. id="registrationForm" digunakan sebagai selektor di JavaScript untuk memasang event listener submit. Atribut novalidate menonaktifkan validasi bawaan browser (seperti popup tooltip merah pada input type="email") sehingga tampilan dan alur validasi sepenuhnya dikendalikan oleh kode JavaScript yang kita tulis sendiri.

c. Input teks, email, dan angka

```html
<div class="form-group">
    <label for="nama">Nama Lengkap</label>
    <input type="text" id="nama" placeholder="Masukkan nama lengkap kamu" />
    <span class="error" id="errNama">⚠ Nama tidak boleh kosong!</span>
</div>

<div class="form-group">
    <label for="email">Email</label>
    <input type="email" id="email" placeholder="contoh@email.com" />
    <span class="error" id="errEmail">⚠ Format email tidak valid!</span>
</div>

<div class="form-group">
    <label for="umur">Umur</label>
    <input type="number" id="umur" placeholder="Masukkan umur kamu" min="1" max="120" />
    <span class="error" id="errUmur">⚠ Umur harus antara 1 - 120!</span>
</div>
```

- ** ** — Pembungkus per field. Setiap form-group berisi satu label, satu input, dan satu pesan error, sehingga ketiganya bisa diatur jarak dan posisinya secara bersamaan dari CSS.
- **<label for="nama">** — Atribut for harus sama dengan id input yang dituju. Ini menghubungkan label ke input secara semantik: saat label diklik, fokus otomatis berpindah ke input yang bersangkutan, meningkatkan usability terutama di perangkat mobile.
- **<input type="text" id="nama">** — Input untuk teks biasa. id="nama" digunakan JavaScript untuk mengambil nilainya via getElementById('nama').
- **<input type="email" id="email">** — Input khusus email. Meskipun novalidate dipasang di form, type="email" tetap berguna untuk menampilkan keyboard yang sesuai di perangkat mobile (memunculkan tombol @ secara otomatis).
- **<input type="number" id="umur" min="1" max="120">** — Input khusus angka. Atribut min dan max mendefinisikan batas nilai yang valid. Nilai ini digunakan ulang di JavaScript saat validasi (umur >= 1 && umur <= 120).
- **** — Elemen inline untuk pesan error. Menggunakan (bukan <p>) agar tidak membuat baris baru. Class "error" di CSS mengatur display:none secara default, sedangkan id unik per field (errNama, errEmail, errUmur) memungkinkan JavaScript menampilkan error secara selektif hanya pada field yang bermasalah.

d. Radio button (jenis kelamin)

```
<div class="form-group">
    <label>Jenis Kelamin</label>
    <div class="radio-group">
    <label><input type="radio" name="gender" value="Laki-laki" /> Laki-laki</label>
    <label><input type="radio" name="gender" value="Perempuan" /> Perempuan</label>
    <label><input type="radio" name="gender" value="Lainnya" /> Lainnya</label>
    </div>
    <span class="error" id="errGender">⚠ Pilih jenis kelamin!</span>
</div>
```

- ** ** — Pembungkus khusus untuk mengatur radio button secara horizontal (flex-direction: row) menggunakan CSS, yang kemudian berubah menjadi vertikal pada layar kecil via media query.
- **name="gender"** — Ketiga radio button menggunakan name yang sama persis. Atribut name inilah yang membuat browser mengelompokkan mereka sebagai satu grup, sehingga hanya satu yang bisa dipilih pada satu waktu. Jika name berbeda, ketiganya bisa dipilih bersamaan.
- **value="Laki-laki"** — Nilai yang dikirim atau dibaca JavaScript saat radio ini dipilih. JavaScript mengambilnya menggunakan querySelector('input[name="gender"]:checked').value.
- **<label><input ... /> Laki-laki</label>** — Radio button dibungkus di dalam label. Ini adalah teknik umum agar area klik label (teks "Laki-laki") juga bisa digunakan untuk memilih radio button, tidak hanya lingkaran kecilnya saja.

e. Dropdown (Jurusan)

```
<div class="form-group">
    <label for="jurusan">Jurusan</label>
    <select id="jurusan">
    <option value="">-- Pilih Jurusan --</option>
    <option value="Teknik Informatika">Teknik Informatika</option>
    <option value="Sistem Informasi">Sistem Informasi</option>
    <option value="Teknik Elektro">Teknik Elektro</option>
    <option value="Manajemen">Manajemen</option>
    <option value="Akuntansi">Akuntansi</option>
    </select>
    <span class="error" id="errJurusan">⚠ Pilih jurusan terlebih dahulu!</span>
</div>
```

- **<select id="jurusan">** — Elemen dropdown yang menampilkan daftar pilihan. id="jurusan" digunakan JavaScript untuk mengambil nilai yang dipilih via getElementById('jurusan').value.
- **<option value="">-- Pilih Jurusan --</option>** — Option pertama sebagai placeholder dengan value kosong ("""). Ini adalah nilai default saat halaman pertama kali dimuat. Validasi JavaScript memeriksa apakah value masih "" (belum dipilih): jika ya, error ditampilkan.
- **<option value="Teknik Informatika">** — Setiap option memiliki atribut value yang merupakan data yang sesungguhnya diambil oleh JavaScript, sedangkan teks di antara tag adalah yang terlihat oleh pengguna. Keduanya bisa berbeda, namun di sini dibuat sama untuk kemudahan.

f. Checkbox (hobi) dan Textarea (pesan)

```html
<div class="form-group">
    <label>Hobi (boleh pilih lebih dari satu)</label>
    <div class="checkbox-group">
    <label><input type="checkbox" name="hobi" value="Membaca" /> Membaca</label>
    <label><input type="checkbox" name="hobi" value="Coding" /> Coding</label>
    <label><input type="checkbox" name="hobi" value="Olahraga" /> Olahraga</label>
    <label><input type="checkbox" name="hobi" value="Musik" /> Musik</label>
    <label><input type="checkbox" name="hobi" value="Gaming" /> Gaming</label>
    </div>
</div>

<div class="form-group">
    <label for="pesan">Pesan / Deskripsi Singkat</label>
    <textarea id="pesan" placeholder="Ceritakan sedikit tentang dirimu..."></textarea>
</div>
```

- **type="checkbox"** — Berbeda dari radio button, checkbox memungkinkan lebih dari satu nilai dipilih secara bersamaan. Meskipun name sama ("hobi"), browser tidak membatasi pilihan.
- **name="hobi"** — Name yang sama pada semua checkbox memungkinkan JavaScript mengambil seluruh yang dipilih sekaligus menggunakan querySelectorAll('input[name="hobi"]:checked'), lalu hasilnya diubah menjadi array dan digabung dengan join(', ').
- **<textarea id="pesan">** — Elemen untuk input teks panjang multi-baris. Berbeda dari <input>, textarea memiliki tag penutup terpisah. Di CSS ditambahkan resize: vertical agar pengguna hanya bisa mengubah tinggi, bukan lebarnya, menjaga layout tetap rapi.
- **Hobi dan Pesan tidak memiliki span.error** — Karena kedua field ini bersifat opsional dan tidak divalidasi. Jika tidak diisi, JavaScript akan menampilkan teks default seperti 'Tidak ada pilihan' atau '(tidak diisi)'.

g. Tombol aksi dan kartu hasil

```html
        <button type="submit" class="btn-submit">Kirim Data</button>
        <button type="button" class="btn-reset" onclick="resetForm()">Reset Form</button>
    </form>
</div>

<div id="result">
    <h2>Data Berhasil Terkirim!</h2>
    <div class="result-grid" id="resultGrid"></div>
</div>
```

- **<button type="submit">** — Tombol dengan type="submit" secara default akan memicu event submit pada form induknya. Event inilah yang ditangkap oleh addEventListener('submit', ...) di JavaScript.
- **<button type="button" onclick="resetForm()">** — Tombol reset menggunakan type="button" yang penting. Jika menggunakan type="submit", tombol ini justru akan mengirim form. Dengan type="button", tidak ada aksi default yang terjadi selain menjalankan fungsi resetForm() yang didefinisikan di JavaScript.
- ** ** — Kontainer kartu hasil yang awalnya disembunyikan oleh CSS (display: none). JavaScript akan mengubah display menjadi 'block' setelah validasi berhasil.

- ** ** — Div kosong yang akan diisi secara dinamis oleh JavaScript menggunakan createElement dan appendChild. id="resultGrid" digunakan sebagai target tempat elemen-elemen hasil dimasukkan.

2. Tampilan CSS (style.css)

a. Background halaman

```
body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: linear-gradient(135deg, #FFF1F2 0%, #EDBDBC 30%, #99B2CF 100%);
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 40px 20px;
  color: #313E57;
}
```

Background menggunakan gradient tiga warna dari palet Whimsical Sky Garden untuk menciptakan kesan langit lembut. Flexbox digunakan agar konten selalu berada di tengah secara horizontal.

b. Card formulir

```
.card {
  background: rgba(255, 255, 255, 0.95);
  backdrop-filter: blur(12px);
  border: 2px solid rgba(237, 189, 188, 0.5);
  border-radius: 24px;
  padding: 40px;
  width: 100%;
  max-width: 620px;
  box-shadow: 0 20px 60px rgba(49, 62, 87, 0.15);
  margin-top: 40px;
}
```

Card menggunakan teknik glassmorphism dengan background semi-transparan dan backdrop-filter blur untuk kesan kaca buram. border-radius: 24px memberikan sudut membulat yang modern.

c. Styling input dan efek fokus

```
input[type="text"],
input[type="email"],
input[type="number"],
select,
textarea {
  width: 100%;
  padding: 13px 16px;
  background: #FFF1F2;
  border: 2px solid #EDBDBC;
  border-radius: 12px;
  color: #313E57;
  font-size: 0.95rem;
  transition: border-color 0.3s, background 0.3s, box-shadow 0.3s;
  outline: none;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}

input:focus, select:focus, textarea:focus {
  border-color: #598099;
  background: #ffffff;
  box-shadow: 0 0 0 4px rgba(89, 128, 153, 0.15);
}
```

Saat field aktif (focus), warna border berubah dari pink ke biru dusty dan muncul efek glow berwarna biru transparan. Transisi CSS 0.3s membuat perubahan tampak halus.

d. Tombol submit dan reset

```
.btn-submit {
  width: 100%;
  padding: 14px;
  background: linear-gradient(135deg, #598099, #313E57);
  color: #ffffff;
  font-size: 1rem;
  font-weight: 700;
  border: none;
  border-radius: 14px;
  cursor: pointer;
  letter-spacing: 1px;
  transition: transform 0.2s, box-shadow 0.2s;
  margin-top: 10px;
}

.btn-submit:hover {
  transform: translateY(-2px);
  box-shadow: 0 8px 25px rgba(89, 128, 153, 0.35);
}

.btn-submit:active {
  transform: translateY(0);
}

.btn-reset {
  width: 100%;
  padding: 12px;
  background: transparent;
  color: #598099;
  font-size: 0.9rem;
  font-weight: 600;
  border: 2px solid #EDBDBC;
  border-radius: 14px;
  cursor: pointer;
  margin-top: 10px;
  transition: border-color 0.3s, color 0.3s, background 0.3s;
}
```

Tombol submit menggunakan gradient diagonal dari Dusty Blue ke Deep Navy Blue. Efek hover mengangkat tombol 2px ke atas dengan translateY dan menambahkan bayangan untuk kesan tombol terangkat.

e. Kartu hasil dan animasi fadeIn

```
#result {
  display: none;
  background: rgba(255, 255, 255, 0.95);
  border: 2px solid #99B2CF;
  border-radius: 20px;
  padding: 30px;
  margin-top: 30px;
  width: 100%;
  max-width: 620px;
  animation: fadeIn 0.5s ease;
  box-shadow: 0 15px 50px rgba(49, 62, 87, 0.12);
}

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(20px); }
  to   { opacity: 1; transform: translateY(0); }
}

#result h2 {
  font-size: 1.3rem;
  color: #313E57;
  margin-bottom: 18px;
  display: flex;
  align-items: center;
  gap: 10px;
}

.result-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 14px;
}
```

Kartu hasil awalnya disembunyikan dengan display: none dan akan muncul dengan animasi fadeIn yang menggeser dari bawah ke posisi normal. Data ditampilkan dalam grid dua kolom agar lebih rapi.

f. Desain responsif (media query)

```
@media (max-width: 600px) {
  .form-header h1 {
    font-size: 1.5rem;
  }

  .form-header p {
    font-size: 0.85rem;
  }

  .card {
    padding: 28px 20px;
    margin-top: 20px;
  }

  .result-grid {
    grid-template-columns: 1fr;
  }

  .radio-group, .checkbox-group {
    flex-direction: column;
    gap: 10px;
  }
}
```

Media query mengubah tata letak untuk layar di bawah 600px: grid hasil menjadi satu kolom, pilihan radio/checkbox disusun vertikal, dan padding card dikurangi agar konten tidak terpotong di layar kecil.

3. Logika JavaScript (script.js)

a. Fungsi showError dan isValidEmail

```
function showError(id, show) {
  const el = document.getElementById(id);
  el.style.display = show ? 'block' : 'none';
}

// — Validasi email dengan regex —
function isValidEmail(email) {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}
```

showError mengontrol visibilitas span error berdasarkan ID elemen. isValidEmail menggunakan ekspresi reguler (regex) untuk memastikan email memiliki format yang benar, yaitu terdapat karakter sebelum @, nama domain, dan ekstensi domain.

b. Fungsi getRadiovalue dan getCheckbocvalues

```
function getRadioValue(name) {
  const selected = document.querySelector(`input[name="${name}"]:checked`);
  return selected ? selected.value : '';
}

// — Ambil nilai checkbox (bisa lebih dari satu) —
function getCheckboxValues(name) {
  const checked = document.querySelectorAll(`input[name="${name}"]:checked`);
  const values = Array.from(checked).map(el => el.value);
  return values.length > 0 ? values.join(', ') : 'Tidak ada pilihan';
}
```

getRadioValue menggunakan pseudo-selector :checked untuk mengambil satu nilai yang dipilih. getCheckboxValues menggunakan querySelectorAll untuk mengambil semua checkbox yang dipilih, lalu digabung menjadi satu string dengan join.

c. Fungsi validateForm

```
function validateForm() {
  let valid = true;

  const nama    = document.getElementById('nama').value.trim();
  const email   = document.getElementById('email').value.trim();
  const umur    = parseInt(document.getElementById('umur').value);
  const gender  = getRadioValue('gender');
  const jurusan = document.getElementById('jurusan').value;

  // Nama
  const namaOk = nama.length > 0;
  showError('errNama', !namaOk);
  if (!namaOk) valid = false;

  // Email
  const emailOk = isValidEmail(email);
  showError('errEmail', !emailOk);
  if (!emailOk) valid = false;

  // Umur
  const umurOk = !isNaN(umur) && umur >= 1 && umur <= 120;
  showError('errUmur', !umurOk);
  if (!umurOk) valid = false;

  // Gender
  const genderOk = gender !== '';
  showError('errGender', !genderOk);
  if (!genderOk) valid = false;

  // Jurusan
  const jurusanOk = jurusan !== '';
  showError('errJurusan', !jurusanOk);
  if (!jurusanOk) valid = false;

  return valid;
}
```

Setiap field diperiksa secara berurutan. Jika satu field tidak valid, flag valid diubah ke false dan pesan error ditampilkan, namun validasi tetap dilanjutkan ke field berikutnya agar semua error muncul sekaligus.

d. Fungsi displayResult

```
function displayResult(data) {
  const grid = document.getElementById('resultGrid');
  grid.innerHTML = '';

  data.forEach(item => {
    const div = document.createElement('div');
    div.classList.add('result-item');
    div.innerHTML = `
      <div class="r-label">${item.label}</div>
      <div class="r-value">${item.value}</div>
    `;
    grid.appendChild(div);
  });

  const resultEl = document.getElementById('result');
  resultEl.style.display = 'block';
```

Fungsi ini membangun kartu hasil secara dinamis menggunakan createElement dan innerHTML. Setelah semua item dirender, kartu ditampilkan dan halaman digulir secara otomatis ke posisi kartu menggunakan scrollIntoView.

e. Fungsi resetForm dan event Submit

```
function resetForm() {
  document.getElementById('registrationForm').reset();
  ['errNama','errEmail','errUmur','errGender','errJurusan'].forEach(id => showError(id, false));
  document.getElementById('result').style.display = 'none';
}

// — Event: submit form —
document.addEventListener('DOMContentLoaded', function() {
  const form = document.getElementById('registrationForm');

  form.addEventListener('submit', function(e) {
    e.preventDefault(); // Cegah reload halaman

    if (!validateForm()) return; // Berhenti jika ada error

    // Kumpulkan data dari form
    const formData = [
      { label: 'Nama Lengkap',  value: document.getElementById('nama').value.trim() },
      { label: 'Email',         value: document.getElementById('email').value.trim() },
      { label: 'Umur',          value: document.getElementById('umur').value + ' tahun' },
      { label: 'Jenis Kelamin', value: getRadioValue('gender') },
      { label: 'Jurusan',       value: document.getElementById('jurusan').value },
      { label: 'Hobi',          value: getCheckboxValues('hobi') },
      { label: 'Pesan',         value: document.getElementById('pesan').value.trim() || '(tidak diisi)' },
      { label: 'Waktu Kirim',   value: new Date().toLocaleString('id-ID') },
    ];

    displayResult(formData);
  });
```

e.preventDefault() mencegah halaman reload saat form dikirim. Event listener dibungkus dalam DOMContentLoaded agar script berjalan setelah semua elemen

HTML tersedia. Data dikumpulkan dalam array objek lalu diserahkan ke displayResult untuk dirender.